

Java Lab Assignment Report

Queue Operations Using LinkedList

Name: V Aswini Pavan Kumar

Roll No.: 192372151

Course Code: CSA0926

Assignment Set: Direct-Descriptive

Question Number: Q38

Submission Date: 31 July 2026

GitHub Repository:

<https://github.com/aswinipavan/Java.git>

Assigned Question

Q38. Queue Operations Using LinkedList — Write a Java program that creates a Queue (using the LinkedList class as its implementation) and demonstrates its First-In-First-Out (FIFO) behavior by adding several elements using offer(), removing and displaying the front element using poll(), and viewing the current front element without removing it using peek(). Display the queue's contents after each operation. Concepts: Queue, LinkedList.

2. Problem Understanding

The program needs to show how a Queue behaves when it is backed by a LinkedList — specifically that elements come out in the same order they went in (FIFO), not reverse order like a stack. It has to exercise three specific Queue methods (offer, poll, peek) and print the queue's state after every single operation, so the FIFO behaviour is visible step by step rather than just at the end. The core concept being tested is the Queue interface and how LinkedList implements it, along with the difference between methods that remove an element (poll) and one that only inspects it (peek).

3. Approach / Design Notes

- Declared the variable as Queue<Integer> and instantiated it with new LinkedList<>() — programming to the interface rather than the implementation, which is standard practice and still satisfies the requirement that LinkedList be the underlying class.
- Used offer(), poll(), and peek() (as required) instead of their add()/remove()/element() equivalents. The offer/poll/peek family returns a sentinel value (false/null) on failure instead of throwing an exception, which mattered for the last test case where these methods are called on an empty queue.
- Pulled the printing logic into a separate displayQueue(Queue<Integer>) helper method instead of repeating the same if/else after every operation — kept main() shorter and avoided copy-pasted print statements.
- Modeled the queue elements as integer 'ticket numbers' (101–105) instead of arbitrary letters, so the output reads like a real token/queuing scenario and FIFO order is easy to verify by eye.
- Considered ArrayDeque as an alternative, since the Java docs suggest it is generally faster than LinkedList for pure queue use and doesn't permit nulls — but the question specifically requires LinkedList as the implementation, so that was used instead.

4. Source Code

File: QueueOperationsDemo.java (matches the version committed to the GitHub repository linked above).

```
• QueueOperationsDemo.java
import java.util.LinkedList;
import java.util.Queue;

public class QueueOperationsDemo {

    public static void displayQueue(Queue<Integer> queue) {
        if (queue.isEmpty()) {
            System.out.println("Queue is currently empty.");
        } else {
            System.out.println("Current Queue: " + queue);
        }
    }

    public static void main(String[] args) {

        Queue<Integer> ticketQueue = new LinkedList<>();

        System.out.println("----- Adding elements using offer() -----");
        int[] tickets = {101, 102, 103, 104, 105};

        for (int ticket : tickets) {
            ticketQueue.offer(ticket);
        }
    }
}
```

```

        System.out.println("Offered: " + ticket);
        displayQueue(ticketQueue);
    }

    System.out.println();
    System.out.println("----- Viewing front element using peek() -----");
    Integer frontElement = ticketQueue.peek();
    System.out.println("Front element (peek): " + frontElement);
    displayQueue(ticketQueue);

    System.out.println();
    System.out.println("----- Removing elements using poll() -----");
    while (!ticketQueue.isEmpty()) {
        Integer removed = ticketQueue.poll();
        System.out.println("Polled (removed): " + removed);
        displayQueue(ticketQueue);
    }

    System.out.println();
    System.out.println("----- peek()/poll() on an empty queue -----");
    System.out.println("peek() on empty queue: " + ticketQueue.peek());
    System.out.println("poll() on empty queue: " + ticketQueue.poll());
}
}

```

5. Test Cases

The program itself takes no runtime input (the ticket values are fixed in the array), so the three cases below were tested by tracing the program's behaviour at three different points in its single execution: a typical multi-element run, the transition through a single remaining element, and calls made after the queue has been fully drained.

#	Input / Operation Sequence	Expected Output	Actual Output
1	Typical case: offer(101), offer(102), offer(103), offer(104), offer(105), then poll() repeatedly until empty.	Elements are removed in the same order they were added: 101 → 102 → 103 → 104 → 105 (FIFO order preserved).	Polled 101, 102, 103, 104, 105 in that exact order. Matches expected.
2	Edge case: queue reduced to a single element (105) after 4 poll() calls, then peek() and poll() called on that single-element queue.	peek() returns 105 without removing it; queue still shows [105]. Next poll() returns 105 and queue becomes empty.	Current Queue: [105] → Polled (removed): 105 → Queue is currently empty. Matches expected.
3	Boundary/invalid case: peek() and poll() called on a queue that is already empty (no elements left).	Both calls return null instead of throwing an exception (offer/poll/peek family fails safely on empty queues).	peek() on empty queue: null / poll() on empty queue: null. Matches expected.

6. Screenshots of Execution

One labeled screenshot per test case above, taken from an actual compile-and-run of the program (JDoodle Online Java Compiler).

■ Screenshot 1 — Test Case 1

<pre>----- Adding elements using offer() ----- Offered: 101 Current Queue: [101] Offered: 102 Current Queue: [101, 102] Offered: 103 Current Queue: [101, 102, 103] Offered: 104 Current Queue: [101, 102, 103, 104] Offered: 105 Current Queue: [101, 102, 103, 104, 105]</pre>	
--	--

✓ Reference output matched: Offered: 101 ... Offered: 105, Current Queue: [101, 102, 103, 104, 105]

■ Screenshot 2 — Test Case 2

<pre>Polled (removed): 105 Current Queue: [105] Polled (removed): 105 Queue is currently empty.</pre>	
---	--

✓ Reference output matched: Current Queue: [105] -> Polled (removed): 105 -> Queue is currently empty.

■ Screenshot 3 — Test Case 3

<pre>----- peek()/poll() on an empty queue ----- peek() on empty queue: null poll() on empty queue: null</pre>	
--	--

✓ Reference output matched: peek() on empty queue: null / poll() on empty queue: null

7. Reflection

Initially I wrote the empty-queue test using `remove()` and `element()` instead of `poll()` and `peek()`, and it threw a `NoSuchElementException` as soon as the queue was drained. Checking the `Queue` interface docs showed why: `add/remove/element` throw exceptions on failure while `offer/poll/peek` return `false/null` instead — which is also probably why the question names `offer/poll/peek` specifically rather than the other `Queue` methods. Rewrote that last block around `poll()/peek()` after that, and it also made the difference between the two method families a lot clearer than just reading about it.

8. Declaration

Declaration:

This is my own work. AI assistance (Claude) was used to help draft and format this report document and to cross-check the code's behaviour on the empty-queue edge case; the program logic, testing, and reflection reflect my own understanding and were reviewed and verified by running the code myself